แบบฝึกหัดปฏิบัติการคาบที่ 7: Polymorphism

คำสั่ง

1. ให้ศึกษาหลักการ Polymorphism ต่อไปนี้
    1. ความสัมพันธ์แบบการสืบทอดคุณสมบัติ อนุญาตให้คลาสสามารถสืบทอดคุณสมบัติจากคลาสแม่(Superclass) ได้
    2. คลาสลูก(Subclass) สามารถเพิ่มคุณสมบัติใหม่ ๆ ได้ โดย Subclass จะเป็นคลาสที่เจาะจงลงไปจากคลาสแม่
    3. ทุก ๆ วัตถุที่สร้างจาก Subclass จะถือว่าเป็น instance ของ superclass ด้วย เช่น
        a. ทุก ๆ วัตถุที่สร้างจากคลาส circle ถือเป็นวัตถุของคลาส Object
        b. แต่ทุก ๆ วัตถุของคลาส Object ไม่ได้เป็นวัตถุชนิด circle
    4. กรณีที่ในเมธอดมีการประกาศรับพารามิเตอร์เป็นแบบ instance ของ superclass เราสามารถส่ง instance ของ subclass ไปเป็นพารามิเตอร์ของ superclass ได้
    5. ตัวแปรอ้างอิงของซูเปอร์คลาสสามารถชี้อินสแตนท์ของซับคลาส
    6. แต่ตัวแปรอ้างอิงของซับคลาสไม่สามารถชี้อินสแตนท์ของซูเปอร์คลาสหรืออินสแตนท์ของคลาสที่มีซูเปอร์คลาสเดียวกันได้ ถ้าจะลองนำไปชี้คลาสอื่นๆที่ไม่มีความเกี่ยวข้องกันเลยยิ่งไม่ได้
    7. การที่ตัวแปรของซูเปอร์คลาสสามารถชี้อินสแตนท์ของซับคลาสได้ เป็นการเพิ่มความยืดหยุ่นในการเขียนโปรแกรม ลักษณะนี้เรียกว่า โพลิมอร์ฟิซึม

Dynamic Binding

- กำหนดให้วัตถุ o เป็นอินสแตนซ์ของคลาส C_1 C_2 … C_{n-1} C_n เมื่อ C_1 เป็นคลาสลูกของ C_2 และ C_2 เป็นคลาสลูกของ C3 … และ C_{n-1} เป็นคลาสลูกของ C_n ดังปรากฏในรูปกล่าวคือ
    - C_n จะเป็นคลาสที่มีความเป็นทั่วไปมากที่สุด(most general class)
    - C_1 เป็นคลาสที่มีความเจาะจงมากที่สุด(most specific class) โดยคลาส C_n ในภาษาจาวาคือคลาส Object
- ถ้า o เป็นอินสแตนซ์ของคลาส C_1 โดยที่ o เรียกเมธอด p แล้ว JVM จะทำการค้นหาเมธอดที่ตรงกับที่ต้องการมากที่สุดโดยจะค้นหาเมธอด p ในคลาส C_1 C_2 … C_{n-1} C_n ตามลำดับจนกว่าจะพบ

Static Binding

คือการผูกตัวแปรอ้างอิงกับชนิดข้อมูลช่วง Compile time

1. ให้ศึกษาและทดลองพิมพ์ Code ตัวอย่างการสืบทอดคลาสจากตัวอย่างต่อไปนี้

``
class A{
   void f(){
      System.out.println("A");
   }
}
class B extends A{
   void f(){
      System.out.println("B");
   }
}
class Test38{
   static void t(A a){
      a.f();
   }
   public static void main(String[] args){
      t(new A());
      t(new B());
   }
}
``
	ให้อธิบายผลลัพธ์ของโปรแกรมพร้อมอธิบายเหตุใดจึงได้คำตอบดังกล่าว
	เกิดจากหลักการ Dynamic Binding ใน Polymorphism ถึงแม้เมธอด `t(A a)` จะรับพารามิเตอร์เป็นประเภท `A` แต่ในรอบที่สองวัตถุจริงที่สร้างส่งเข้าไปคือ `new B()` ซึ่งเป็น Subclass เมื่อเรียก `a.f()` ตัว JVM จึงเลือกเรียกใช้เมธอด `f()` ของคลาส `B` ที่ถูก Overriding ไว้

``
class A{
   int x=1;
   void inc(){x++;}
   int get(){return x;}
}
class B extends A{
   int x=2;
   void inc(){x++;}
   int get(){return x;}
}
class Test{
  public static void main(String[] args){
``
	ให้อธิบายถึงส่วนที่มีการทำงานเป็น Static binding **และ** Dynamic binding
	Static binding: เกิดขึ้นกับการเข้าถึงตัวแปร (Field) `a.x` และ `b.x` การผูกตัวแปรจะอ้างอิงตามประเภทของตัวแปร ณ ช่วง Compile time ทำให้ `a.x` ดึงค่า `x` ของคลาส `A` ส่วน `b.x` ดึงค่า `x` ของคลาส `B`

B b =new B();
A a= b;
a.inc();
b.inc();
System.out.println(a.x+","+b.x);
System.out.println(a.get()+","+b.get());
}
}
	Dynamic binding: เกิดขึ้นกับการเรียกใช้เมธอด `a.get()` และ `b.get()` JVM จะค้นหาเมธอดจากวัตถุจริง ณ เวลา Runtime ซึ่งวัตถุจริงของทั้ง `a` และ `b` คือ `B` เมธอด `get()` ที่ถูกเรียกจึงเป็นของคลาส `B` ทั้งคู่ คืนค่า `x` ของ `B`

public class Test{
public static void main(String[] args){
Object fruit = new Fruit();
Object apple= new (Apple)fruit;
}
}
class Apple extends Fruit{ }
class Fruit{}
	จงอธิบายส่วนที่ผิดของโปรแกรม
	บรรทัด `Object apple = new (Apple)fruit;` ผิดไวยากรณ์ (Syntax Error) เนื่องจากวางเครื่องหมาย Casting `(Apple)` ไว้หลังคีย์เวิร์ด `new`

2. จงอธิบายผลลัพธ์ของ Output ต่อไปนี้

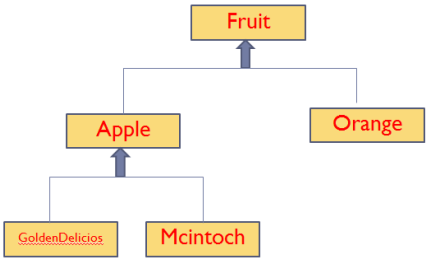
public class Test {
public static void main(String[] args) {
Object a1 = new A();
Object a2 = new Object();
System.out.println(a1);
System.out.println(a2);
}
}
class A {
int x;
public String toString() {
return "A's x is " + x;
}
}
	`A's x is 0`
	`java.lang.Object@15db9742`
	
	บรรทัดแรกเรียก `toString()` ของคลาส `A` ที่โดน Overriding ส่วนบรรทัดที่สองเรียก `toString()` ของคลาส `Object`

public class Test {
public static void main(String[] args) {
Object a1 = new A();
Object a2 = new A();
System.out.println(a1.equals(a2));
}
}
class A {
int x;
public boolean equals(A a) {
	`false`
	
	เป็นเพียงการ Overload ไม่ใช่ Overriding ของ `equals(Object obj)` เมื่อเรียก `a1.equals(a2)` JVM จึงไปเรียก `equals` ของ

`return this.x == a.x;`
`}`
`}`
	คลาส `Object` ซึ่งเปรียบเทียบจาก Memory Address จึงคืนค่า `false`

`public class Test {`
`public static void main(String[] args) {`
`A a1 = new A();`
`A a2 = new A();`
`System.out.println(a1.equals(a2));`
`}`
`}`
`class A {`
`int x;`
`public boolean equals(A a) {`
`return this.x == a.x;`
`}`
`}`
	`true`
	
	ตัวแปร `a1` และ `a2` ถูกประกาศประเภทเป็น `A` โดยตรง ทำให้เรียกใช้เมธอด `equals(A a)` ของคลาส `A` ซึ่งเปรียบเทียบค่า `this.x == a.x` (0 == 0) จึงคืนค่า `true`

3. กำหนดให้ Fruit, Apple, Orange, Golden Delicious Apple, และ Macintosh Apple มีโครงสร้างการสืบทอดและการประกาศตัวแปรดังนี้ ให้เขียนโปรแกรมเพื่อจำลองการทำงานของโครงสร้างของคลาสดังกล่าวพร้อมตอบคำถามต่อไปนี้



กำหนดการทำงานภายในmain() มีการสร้าง Object ดังนี้

```
Fruit fruit = new GoldenDelicious();
Orange orange = new Orange() ;
```